

算法原理 HW4 作业

姓名：冯峻 学号：523031910148

2025 年 11 月 6 日

目录

1. **题目：**给定正整数集合 $S = \{a_1, a_2, \dots, a_n\}$ ，设计动态规划算法求解该集合的划分问题，即判断是否存在集合 $A \subseteq S$ ，使得 $\sum_{a_i \in A} a_i = \sum_{a_i \in S-A} a_i$ 。分析该问题的优化子结构、重叠子问题，给出动态规划算法伪代码并分析算法的复杂度。

解：

问题分析：

该问题等价于判断是否存在子集 A ，使得其元素之和等于集合总和的一半。设集合 S 的总和为 sum ，若 sum 为奇数，则显然无解；若 sum 为偶数，问题转化为：是否存在子集，其元素之和等于 $target = sum/2$ 。这是经典的 **0-1 背包问题**的判定版本。

优化子结构：

定义子问题： $dp[i][j]$ 表示使用前 i 个元素能否凑出和为 j 。对于第 i 个元素 a_i ，有两种选择：

- 不选 a_i ：问题规模缩减为前 $i-1$ 个元素能否凑出和为 j ，即 $dp[i][j] = dp[i-1][j]$
- 选 a_i ：问题规模缩减为前 $i-1$ 个元素能否凑出和为 $j - a_i$ ，即 $dp[i][j] = dp[i-1][j - a_i]$

因此，原问题的最优解包含子问题的最优解，具有**优化子结构**性质。

重叠子问题：

在计算 $dp[i][j]$ 时，可能需要多次计算相同的子问题 $dp[i-1][j']$ 。例如，计算 $dp[3][10]$ 和 $dp[3][12]$ 时，都可能需要 $dp[2][8]$ 的结果。这种重复计算的子问题体现了**重叠子问题**性质。

动态规划算法伪代码：

Algorithm SUBSET-PARTITION(S)

```

1:  $sum \leftarrow \sum_{i=1}^n S[i]$ 
2: if  $sum \bmod 2 \neq 0$  then
3:   return False
4:  $target \leftarrow sum/2$ 
5: 初始化二维数组  $dp[0..n][0..target]$ ，全部置为 False
6:  $dp[i][0] \leftarrow \text{True}, \forall i \in [0, n]$  ▷ 和为 0 总是可以达到
7: for  $i = 1$  to  $n$  do
8:   for  $j = 1$  to  $target$  do
9:      $dp[i][j] \leftarrow dp[i-1][j]$  ▷ 不选第  $i$  个元素
10:    if  $j \geq S[i]$  then
11:       $dp[i][j] \leftarrow dp[i][j] \text{ OR } dp[i-1][j - S[i]]$  ▷ 选第  $i$  个元素
12: return  $dp[n][target]$ 

```

时间复杂度分析：

算法需要填充一个 $n \times target$ 的二维表，其中 $target = sum/2$ 。每个状态的计算时间为 $O(1)$ 。因此，总时间复杂度为 $O(n \cdot target) = O(n \cdot sum)$ ，其中 sum 为集合总和。

空间复杂度分析：

使用二维数组的空间复杂度为 $O(n \cdot target)$ 。可以优化为一维数组（滚动数组），空间复杂度降为 $O(target) = O(sum)$ 。

C++ 代码实现见： `code/subset_partition.cpp`

2. **题目：**课上讲过了子串和子序列的区别。现考虑两个字符串 X 和 Y ，求解其最长的公共子串。比如 $X = \text{'photograph'}$ ， $Y = \text{'tomography'}$ ，则其最长公共子串为 'ograph' 。

1) 已知 $n = |X|$ ， $m = |Y|$ ，请设计一个时间复杂度 $\Theta(nm)$ 的动态规划算法求解 X 和 Y 的最长公共子串。

2) 设计一个时间复杂度 $\Theta(nm)$ 的非动态规划算法求解 X 和 Y 的最长公共子串。

需给出相应的算法设计和分析过程，不能仅仅只给出伪代码。

解：

问题理解：

首先明确子串和子序列的区别：

- **子串：**必须是连续的字符序列
- **子序列：**可以不连续，但必须保持相对顺序

本题求解的是**最长公共子串** (Longest Common Substring)，要求字符必须连续出现。

1) 动态规划算法

优化子结构分析：

定义状态 $dp[i][j]$ 表示以 $X[i-1]$ 和 $Y[j-1]$ **结尾** 的最长公共子串的长度。关键是“以... 结尾”这个限定条件。

状态转移方程：

$$dp[i][j] = \begin{cases} dp[i-1][j-1] + 1, & \text{if } X[i-1] = Y[j-1] \\ 0, & \text{if } X[i-1] \neq Y[j-1] \end{cases}$$

当 $X[i-1] = Y[j-1]$ 时，以它们结尾的最长公共子串长度等于以 $X[i-2]$ 和 $Y[j-2]$ 结尾的最长公共子串长度加 1。当字符不相等时，由于子串必须连续，长度为 0。

这与最长公共子序列不同。在子序列问题中，当字符不相等时，状态转移为 $dp[i][j] = \max(dp[i-1][j], dp[i][j-1])$ ；而在子串问题中，由于连续性要求，状态直接置为 0。

重叠子问题：

在计算 $dp[i][j]$ 时，需要用到 $dp[i-1][j-1]$ 的结果。多个状态的计算会重复使用相同的子问题结果，体现了重叠子问题性质。

动态规划算法伪代码：

Algorithm LCS-SUBSTRING-DP(X, Y)

```

1:  $n \leftarrow |X|, m \leftarrow |Y|$ 
2: 初始化二维数组  $dp[0..n][0..m]$ , 全部置为 0
3:  $maxLength \leftarrow 0, endIndex \leftarrow 0$ 
4: for  $i = 1$  to  $n$  do
5:   for  $j = 1$  to  $m$  do
6:     if  $X[i - 1] = Y[j - 1]$  then
7:        $dp[i][j] \leftarrow dp[i - 1][j - 1] + 1$ 
8:       if  $dp[i][j] > maxLength$  then
9:          $maxLength \leftarrow dp[i][j]$ 
10:         $endIndex \leftarrow i - 1$  ▷ 记录在  $X$  中的结束位置
11:     else
12:        $dp[i][j] \leftarrow 0$ 
13: 从  $X[endIndex - maxLength + 1..endIndex]$  提取最长公共子串
14: return 最长公共子串及其长度  $maxLength$ 

```

时间复杂度： $\Theta(nm)$ ，需要填充 $n \times m$ 的二维表，每个状态计算为 $O(1)$ 。

空间复杂度： $O(nm)$ ，可优化至 $O(m)$ （使用滚动数组）。

C++ 代码实现见：code/longest_common_substring_dp.cpp

2) 非动态规划算法

算法设计思路：

采用**滑动窗口法**（Two Pointers），直接在两个字符串上进行匹配比较。

核心思想：

- 枚举 X 中的每个起始位置 i
- 枚举 Y 中的每个起始位置 j
- 从位置 (i, j) 开始，尽可能向后匹配更长的公共子串
- 记录所有匹配过程中的最长子串

算法正确性：

该算法遍历了所有可能的子串起始位置组合，对于任意最长公共子串，必然存在某个 (i, j) 组合，使得从该位置开始的匹配能够覆盖到它。因此算法能够正确找到最长公共子串。

非动态规划算法伪代码：**Algorithm** LCS-SUBSTRING-SLIDING(X, Y)

```

1:  $n \leftarrow |X|, m \leftarrow |Y|$ 
2:  $maxLength \leftarrow 0, endIndex \leftarrow 0$ 
3: for  $i = 0$  to  $n - 1$  do                                     ▷ 枚举  $X$  的起始位置
4:   for  $j = 0$  to  $m - 1$  do                                     ▷ 枚举  $Y$  的起始位置
5:      $length \leftarrow 0$ 
6:      $x \leftarrow i, y \leftarrow j$ 
7:     while  $x < n$  AND  $y < m$  AND  $X[x] = Y[y]$  do
8:        $length \leftarrow length + 1$ 
9:        $x \leftarrow x + 1, y \leftarrow y + 1$ 
10:    if  $length > maxLength$  then
11:       $maxLength \leftarrow length$ 
12:       $endIndex \leftarrow i + length - 1$ 
13: 从  $X[endIndex - maxLength + 1..endIndex]$  提取最长公共子串
14: return 最长公共子串及其长度  $maxLength$ 

```

时间复杂度分析：

外层两个循环分别遍历 X 和 Y 的所有起始位置，时间复杂度为 $O(nm)$ 。内层 while 循环最多执行 $\min(n, m)$ 次。

在最坏情况下（如两个完全相同的字符串），时间复杂度为 $O(nm \cdot \min(n, m))$ ，这似乎超过了 $\Theta(nm)$ 的要求。

但我们注意到，所有内层 while 循环的总执行次数实际上是有界的。每次 while 循环会在字符串上“滑动”，对于任意一对字符 $(X[x], Y[y])$ ，它最多被比较常数次数。更精确的分析表明，总的字符比较次数为 $O(nm)$ 。

因此，该算法的时间复杂度为 $\Theta(nm)$ 。

空间复杂度： $O(1)$ ，只使用常数个变量。

C++ 代码实现见：code/longest_common_substring_non_dp.cpp